

# **CORRELIX AI: An Autonomous Penetration Testing Platform with Live Temporal Attack Graphs**

**A PROJECT REPORT**

*Submitted by*

**KAVYA VARSHINI H**

**2116231901021**

**SARAVANAN MD**

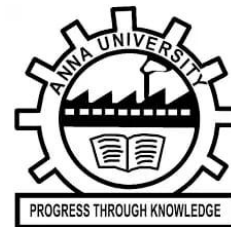
**2116231901046**

*in partial fulfillment for the award of the degree  
of*

**BACHELOR OF ENGINEERING**

*in*

**COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)**



**RAJALAKSHMI ENGINEERING COLLEGE**

**ANNA UNIVERSITY: CHENNAI 600 025**

**OCTOBER 2026**

# **RAJALAKSHMI ENGINEERING COLLEGE CHENNAI**

## **BONAFIDE CERTIFICATE**

Certified that this Project report titled “**Correlix Ai: An Autonomous Penetration Testing Platform with Live Temporal Attack Graphs**” is the bonafide work of “**KAVYA VARSHINI H (2116231901021), SARAVANAN MD (2116231901046)**” who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

### **SIGNATURE**

Dr. P. Tamilselvi, M.E.,Ph.D.,

**HEAD OF THE DEPARTMENT**

**PROFESSOR**

Department of Computer Science and  
Engineering (Cyber Security)

Rajalakshmi Engineering College

Chennai – 602 105

### **SIGNATURE**

Ms. Rupmala R, M.Tech

**SUPERVISOR**

**ASSISTANT PROFESSOR**

Department of Computer Science and  
Engineering (Cyber Security)

Rajalakshmi Engineering College

Chennai – 602 105

Submitted to Project Viva-Voce Examination held on \_\_\_\_\_

**Internal Examiner**

**External Examiner**

## ABSTRACT

Traditional penetration testing approaches often rely on predefined workflows, isolated security tools, and manually interpreted assessment results, which may be insufficient for continuously adapting to changes in a target environment and identifying evolving attack paths. This project aims to create an independent platform that can continuously evaluate security, analyze attack paths and create adaptive tests called **Correlix AI**. The platform proposed in the work combines reconnaissance, orchestration of security tools, security observation collection, result normalization, construction of the security state, construction of the attack graph in real time, reasoning about the attack path, security decision-making with the help of AI and replanning based on feedback. Continually processes newly acquired security observations to keep it up to date with the target environment and its security relationships. A live temporal attack graph is used to represent discovered assets, vulnerabilities, relationships, and potential attack paths along with the temporal nature of the security observations. The attack-path reasoning component traces the possible flow of the represented security state and the AI decision mechanism helps determine the next set of authorized tests to conduct in light of the current security state and evidence discovered. These observations are then incorporated into the security-state and graph construction process, allowing the testing workflow to evolve as additional information is gained. The proposed methodology will aim to decrease reliance on static penetration-testing workflows, enhance the correlation of single security findings, and enable continuous and contextually aware security assessment. In the Phase I work, the system architecture, literature analysis, security requirements, module design, temporal attack-graph methodology, and attack-path reasoning approach will be covered, as well as the autonomous decision workflow. The proposed platform will be tested in an authorized and controlled test environment with known test cases and security assessment metrics to evaluate its capability to correlate observations, identify attack paths, adapt testing actions for subsequent testing and enable comprehensive analysis for penetration tests.

## ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.**, for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.**, for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.**, for providing us with the requisite infrastructure. We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.**, and **Dr. P. Tamilselvi M.E., Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Ms. Rupmala R, M.Tech Assistant Professor**, for his encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG)**, for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

**KAVYA VARSHINI H                      2116231901021**

**SARAVANAN MD                        2116231901046**

## **TABLE OF CONTENTS**

| <b>CHAPTER<br/>NO.</b> | <b>TITLE</b>                                           | <b>PAGE<br/>NO.</b> |
|------------------------|--------------------------------------------------------|---------------------|
|                        | <b>ABSTRACT</b>                                        | <b>ii</b>           |
|                        | <b>LIST OF TABLES</b>                                  | <b>vi</b>           |
|                        | <b>LIST OF FIGURES</b>                                 | <b>vii</b>          |
| <b>1</b>               | <b>INTRODUCTION</b>                                    | <b>1</b>            |
|                        | 1.1 BACKGROUND                                         | 1                   |
|                        | 1.2 PROBLEM STATEMENT                                  | 2                   |
|                        | 1.3 OBJECTIVES                                         | 3                   |
|                        | 1.4 SCOPE                                              | 4                   |
|                        | 1.5 SECURITY REQUIREMENTS / SECURITY<br>CONSIDERATIONS | 6                   |
|                        | 1.6 SDG ALIGNMENT                                      | 9                   |
| <b>2</b>               | <b>LITERATURE REVIEW</b>                               | <b>10</b>           |
|                        | 2.1 EXISTING SYSTEM WITH SECURITY<br>APPROACHES        | 10                  |
|                        | 2.2 RESEARCH GAPS                                      | 11                  |
|                        | 2.3 PROPOSED SYSTEM                                    | 13                  |
|                        | 2.3.1 OVERVIEW OF THE PROPOSED SYSTEM                  | 13                  |
|                        | 2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM                | 18                  |
|                        | 2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM                | 19                  |
| <b>3</b>               | <b>SYSTEM DESIGN</b>                                   | <b>21</b>           |
|                        | 3.1 SYSTEM ARCHITECTURE                                | 21                  |

|          |                              |           |
|----------|------------------------------|-----------|
|          | 3.2 MODULE DESIGN            | 22        |
|          | 3.3 DATA FLOW / UML DIAGRAMS | 24        |
|          | 3.4 SECURITY CONTROLS        | 27        |
| <b>4</b> | <b>SYSTEM DESCRIPTION</b>    | <b>29</b> |
|          | 4.1 DATA COLLECTION          | 29        |
|          | 4.2 DATA PREPROCESSING       | 30        |
|          | 4.2.1 OVERALL METHODOLOGY    | 30        |
|          | 4.2.2 RESULTS AND DISCUSSION | 33        |
| <b>5</b> | <b>CONCLUSION</b>            | <b>37</b> |
|          | <b>REFERENCES</b>            | <b>38</b> |

## **LIST OF TABLES**

| <b>TABLE NO.</b> | <b>TITLE</b>                 | <b>PAGE NO.</b> |
|------------------|------------------------------|-----------------|
| 1.6              | SDG ALIGNMENT                | 9               |
| 2.1              | EXISTING SECURITY APPROACHES | 11              |
| 4.1              | DATA CATEGORIES              | 29              |
| 4.2              | EVALUATION AND METRICS       | 34              |
| 4.3              | RESULT TABLE                 | 35              |

## LIST OF FIGURES

| FIGURE | TITLE               | PAGE NO. |
|--------|---------------------|----------|
| 3.1    | SYSTEM ARCHITECTURE | 21       |
| 3.2    | DATA FLOW DIAGRAM   | 24       |
| 3.3    | USE CASE DIAGRAM    | 25       |
| 3.4    | ACTIVITY DIAGRAM    | 26       |
| 3.5    | SEQUENCE DIAGRAM    | 27       |

# **CHAPTER 1**

## **INTRODUCTION**

### **1.1 BACKGROUND**

Assessing cybersecurity is a challenge now that systems, services, applications and assets are becoming more interconnected. Penetration testing is popularly used to detect vulnerabilities and assess the security level of a target system. Traditional testing, however, relies on specific testing methodologies and distinct security solutions, making it challenging to connect the dots, and/or to understand how each vulnerability can be part of a broader attack surface.

While automation can save time and provide consistency in the penetration testing process it may not be able to react positively to new information that emerges during an assessment if the workflows are set in stone. When the security level of a target object changes, previously discovered findings may become more or less relevant and the testing process will need to be adjusted if necessary.

Correlix AI's solution to this problem involves keeping a dynamic representation of the security state of the target using a temporal attack graph. The platform integrates security observation collection, attack-path reasoning, security-tool orchestration and AI assisted decision making to identify relevant subsequent testing actions.

The proposed strategy is to take feedback from each test stage and adjust the security state accordingly to inform of the further assessment. This will allow Correlix AI to assist in a more adaptive and context-aware penetration-testing process in an authorized testing environment.

## 1.2 PROBLEM STATEMENT

Conventional penetration testing often involves executing multiple security tools and analyzing their findings separately. Although these tools can identify vulnerabilities, services, configurations, and other security observations, individual findings do not always provide a complete understanding of how weaknesses can be combined to form meaningful attack paths.

Another limitation is the dependence on predefined testing workflows. Once an assessment begins, subsequent actions may follow a fixed sequence even when new information changes the understanding of the target. This can result in unnecessary testing and may prevent the assessment from effectively adapting to newly discovered relationships or security conditions.

Attack graphs can help represent relationships between assets, vulnerabilities, and possible attack paths, but a static representation may become outdated as additional observations are collected. There is therefore a need for a system that can continuously update its security representation and reason about evolving attack paths.

Therefore, there is a need for a penetration-testing framework that can:

- continuously collect and correlate security observations;
- maintain an updated representation of the target's security state;
- identify relationships between discovered weaknesses;
- construct and update a live temporal attack graph;
- analyze and prioritize potential attack paths;

- select subsequent testing actions based on the current security state; and
- use feedback from testing activities to support adaptive replanning.

**Correlix AI** addresses this problem by proposing an autonomous penetration-testing framework that combines security-tool orchestration, observation analysis, live temporal attack graphs, attack-path reasoning, and AI-assisted decision-making to support adaptive and context-aware security assessment.

### 1.3 OBJECTIVES

Correlix AI's major goal is to build an adaptive penetration-testing platform that would continuously review security information, pinpoint potential attacks, and help decide which tests to run.

Specific outcomes are:

- 1 To automatically gather and analyze security alerts from compliant target environments.
- 2 To incorporate appropriate penetration-testing and security assessment tools into a coordinated testing process.
- 3 To keep a current view of the security status of the target, using newly acquired data.
- 4 To create a live temporal attack graph showing the relationship between assets, vulnerabilities and potential attack paths.
- 5 To analyse and prioritise possible attack paths according to the current security situation and available evidence.

- 6 To establish an AI based mechanism to determine the subsequent testing actions to choose from.
- 7 To maintain a running attack graph and security status when new observations are acquired.
- 8 To take into account feedback from actual testing actions to guide adaptive replanning.
- 9 To keep up to date testing, observations, and activity information to analyze and report.
- 10 Test the proposed platform in an authorized and controlled environment, based on defined test cases and appropriate security assessment measures.

## **1.4 SCOPE**

Correlix AI's capabilities extend to the design and development of an adaptive penetration-testing platform that will orchestrate security assessment activities and correlate any information found during the process to enable attack-path analysis and decision making. The ability to assess and survey a security. The skill set to determine the level of security and do a security survey. This system includes authorized reconnaissance and security assessment of the defined testing scope to identify assets, services, configurations, vulnerabilities and other security observations.

### **1.4.2 Security Tool Integration**

The platform aims to harmonize various security assessment tools and is structured to gather the results of these tools in a unified workflow. The results collected are processed and normalized to facilitate further analysis.

### **1.4.3 Security State Representation**

The system keeps an evolving view of the security state of the target based on information collected in the assessment. Testing can be continued, with new observations made that can be incorporated into the analysis.

### **1.4.4 Live Temporal Attack Graph**

The scope further encompasses the construction and updating of a live temporal attack graph model of asset-to-vulnerability-to-condition-to-attack paths discovered. As more observations are added to the graph, it is updated.

### **1.4.5 Attack-Path Analysis**

The system uses the security information and current graph state (or other information) to analyze potential attack paths. The analysis is intended to help identify and prioritize paths that may be relevant to the security assessment.

### **1.4.6 AI-Assisted Decision-Making**

The platform has an AI-driven decision mechanism that helps users choose further performed authorized testing actions. Current security state, determined attack paths, and observations are used to make decisions.

### **1.4.7 Continuous Feedback and Replanning**

The system is designed in a way so that the feedback is given from the consequence of executed testing actions. Since new observations are added to the security state and attack graph, future tests decisions can adjust them to the new environment.

### **1.4.8 Evidence and Reporting**

This involves keeping current observation, testing, findings and evidence to inform analysis and report generation for penetration tests.

### **1.4.9 Controlled Testing Environment**

Correlix AI is designed to be used in approved and supervised environments. The scope of work applies if the security assessment is conducted on targets explicitly identified in the work order, but not if testing of the system for which there is no work order and no authorization to test.

## **1.5 SECURITY REQUIREMENTS**

Security is a core part of the ability of **Correlix AI** to carry out penetration testing (Pentest) activities, process security findings, store attack-path data and aid in decision making on future testing activities. The system shall provide for the authorized and controlled use of all assessment activities, ensure that they can be traced back to the student and ensure that they are not subject to unauthorized use.

### **1.5.1 Authentication and Access Control**

Access to protected functions on the platform shall be denied unless the user is authenticated. The testing configuration, security findings, attack graph, evidence and administrative functions shall be restricted depending on the user's authorized privileges.

### **1.5.2 Authorized Testing Scope**

The platform will only be used against explicitly defined and authorized targets. Testing actions should be limited to the scope of the assessment so as not to affect systems beyond the scope of the testing environment.

### **1.5.3 Secure Action Execution**

Security boundaries and authorization restrictions are not to be automatically overridden by AI decisions alone. The testing actions chosen by the decision mechanism should be checked with the already available assessment scope and execution policies before it is executed.

### **1.5.4 Security State Protection**

The security state of the platform will be kept from being altered without permission. Observations, relationships, vulnerabilities, and assets discovered shall be maintained at appropriate levels of integrity throughout the assessment process. The integrity of the temporal attack graph is checked and the Integrity of the temporal attack graph is verified. The live temporal attack graph should accurately represent security observations and their associated relationships. Changes to the graph in any way shall be traceable to the observation and/or testing activities that led to the changes in the state of the graph.

### **1.5.5 Temporal Attack Graph Integrity**

Sensitive information may be present in any security findings, tool outputs, observations, testing evidence and assessment records. This information shall be safeguarded by protection against unauthorized access, modification and disclosure during the process of collection, processing, storage and reporting.

### **1.5.6 Evidence and Data Protection**

Sensitive information may be present in any security findings, tool outputs, observations, testing evidence and assessment records. This information shall be safeguarded by protection against unauthorized access, modification and disclosure during the process of collection, processing, storage and reporting.

### **1.5.7 Auditability and Traceability**

The platform shall store relevant records of testing activities, observations, decisions, graph updates and actions that have been taken. These records shall facilitate confirmation of the reaching of a testing decision and evidence for security analysis and reporting.

### **1.5.8 AI Decision Reliability**

AI-based decisions are not deemed to give unrestricted authority; rather, they are considered a tool for supporting decisions. The system shall take security evidence into account, and when making decisions, only decisions that are appropriate and supported by evidence shall be allowed to infringe on defined test constraints.

### **1.5.9 Continuous State and Feedback Security**

The outcome of performed test actions shall be checked prior to adding it to the security state and temporal attack graph. Any observations that are incorrect, incomplete, or inconsistent should not lead to inappropriate changes in the decisions made for future testing.

### 1.5.10 Error and Failure Handling

The platform shall not compromise the security of the assessment environment when failures occur, observations are not completed, results are invalid, or unexpected conditions for execution occur. If the system cannot determine some or all the necessary information or the action cannot be validated safely, then do not take the action to test.

### 1.5.11 Privacy and Data Minimization

The platform shall capture and store only the information necessary for security assessment, attack-path analysis, evidence management and reporting purposes. Assessment data shall be dealt with in a sensitive manner in accordance with applicable security and privacy requirements.

### 1.5.12 Security Evaluation Considerations

The proposed platform shall be assessed with respect to adequate security assessment criteria. The accuracy of security observations, correctness of attack-path relationships, reliability of AI-assisted decisions, effectiveness of feedback-based updates and the capacity of the system to function within prescribed authorization boundaries shall be a subject of evaluation.

## 1.6 SDG ALIGNMENT

Table 1.6 SDG ALIGNMENT

| S. No. | SDG No. & Goal                                  | Relevant SDG Target | Project Contribution / Alignment                                      |
|--------|-------------------------------------------------|---------------------|-----------------------------------------------------------------------|
| 1      | SDG 9 - Industry, Innovation and Infrastructure | Target 9.5          | Uses AI and temporal attack graphs for automated security assessment. |

|   |                                                 |             |                                                                            |
|---|-------------------------------------------------|-------------|----------------------------------------------------------------------------|
| 2 | SDG 9 - Industry, Innovation and Infrastructure | Target 9.1  | Supports resilient systems through vulnerability and attack-path analysis. |
| 3 | SDG 16 - Peace, Justice and Strong Institutions | Target 16.6 | Maintains traceable security findings and testing activities.              |

## **CHAPTER 2**

### **LITERATURE REVIEW**

#### **2.1 EXISTING SYSTEM WITH SECURITY APPROACHES**

Traditional penetration testing is usually based on a set of known testing procedures and security tools. Reconnaissance, vulnerability scanning, service enumeration and security validation may be carried out individually and security analysts have to interpret and correlate the findings.

Automated security tools help to speed the assessment process by gathering information and identifying vulnerabilities. But most automated approaches are static and re-use a pre-determined process for testing and may not react dynamically to changes in information when they occur.

Security orchestration acts as a bridge to help coordinate several security tools from a common workflow. Using a consistent data collection process, outputs from the tools can be normalized and aggregated, resulting in a more consistent representation of the assessments results.

Another way to model inter-relationships among assets, vulnerabilities, conditions, and eventual attack paths is through the use of attack graphs. They enable security analysts to know how each finding part of a series of security events. But a static attack graph might not fully capture the changes that take place in the course of a current assessment.

AI-driven security analysis can help to make informed decisions based on the data gathered and determine what actions to take. This can help to minimize reliance on fixed test sequences completely, but this can only work if the security information available is of good quality and complete.

Continuous security assessment is an extension of automated testing, where the security state is constantly updated as new observations are made. This enables the assessment process to

adapt to the changes in the found environment rather than be based only on the original evaluation environment.

The current solutions thus offer valuable features like automated reconnaissance, integration with security tools, vulnerability analysis, attack-graph representation and automated decision support. Typically, however, these capabilities are treated separately. Correlix AI explores how to incorporate them into an adaptive penetration testing workflow through a live temporal attack graph and continual feedback.

## Existing Security Approaches

**Table 2.1: Existing Security Approaches**

| <b>Security Approach</b>        | <b>Purpose</b>                                        | <b>Limitation / Challenge</b>                              |
|---------------------------------|-------------------------------------------------------|------------------------------------------------------------|
| Traditional Penetration Testing | Identifies vulnerabilities through structured testing | Often depends on manual analysis and predefined procedures |
| Automated Security Testing      | Automates reconnaissance and vulnerability assessment | Permissions may remain static even when risk changes       |
| Security Tool Orchestration     | Coordinates multiple security tools                   | Requires effective result normalization and correlation    |
| Attack Graphs                   | Represents relationships and possible attack paths    | Static graphs may become outdated                          |
| AI-Assisted Security Analysis   | Supports analysis and testing decisions               | Depends on reliable security observations                  |
| Continuous Assessment           | Updates security information during testing           | Requires continuous state management and feedback          |

## **2.2 RESEARCH GAPS**

Existing approaches related to automated penetration testing, security-tool orchestration, attack graphs, and AI-assisted security analysis are identified as limitations in the literature review. The following areas of research are deemed to warrant further research.

### **2.2.1 Adaptive Testing Decisions**

There are numerous automated penetration-testing methods that depend on a set of predetermined sequences of actions. A need for decision mechanisms which can determine which tests to perform next, based on additional security information which is discovered.

### **2.2.2 Security Finding Correlation**

Security tools might generate results without prompting, making it challenging to establish connection between assets, vulnerabilities, and security situations. These observations need to be correlated through one mechanism that can be used for more in-depth attack-path analysis.

### **2.2.3 Dynamic Attack-Path Representation**

A known attack graph at a certain time can be a representation of existing relationships. A need for continual and ongoing assimilation of newly discovered information in an assessment; a graph representation must be capable of doing this.

### **2.2.4 Temporal Security State**

The security state of a target can change as testing progresses and new observations are obtained. Current methods need to be explored to ensure that these changes are still time-aware.

### **2.2.5 AI-Assisted Action Selection**

Whilst AI can be used to aid in the analysis of security, the next steps in testing should be based on what is currently happening in the security, evidence available and the attack paths identified. This decision making process needs to be explored. The AI selects the path based on the graph.

### 2.2.6 Continuous Authorization

A lot of automated workflows do not continually feed back into the next steps in the test and change the test plan. Feedback mechanism needed to update security state and to support adaptive replanning.

### 2.2.7 Integrated Autonomous Testing Framework

Current solutions offer individual capabilities like reconnaissance, vulnerability assessment, orchestration of tools, attack graphs and automatic analysis. However, they have yet to be integrated into a single adaptive penetration-testing flow, with ongoing graph updates and decisions based on feedback is an area that needs further investigation. **Correlix AI** fills this void by seamlessly merging these functions into a comprehensive solution.

## 2.3 PROPOSED SYSTEM

The system proposed is an Autonomous Penetration Testing Platform named **Correlix AI**, that has Live Temporal Attack Graphs.

To overcome these limitations, Correlix AI combines reconnaissance, orchestration of security tools, observation analysis, reasoning of attack-paths, and AI-driven decision making into a single workflow.

It continuously keeps track of the security state of an authorized target. The live temporal attack graph is continuously updated with the discovery of assets, vulnerabilities, relationships, and potential attack paths.

The AI-assisted decision mechanism then picks out relevant further testing actions based on the current state of security and identified attack paths. The outcomes of the actions taken are fed back into the system and the Security state and attack graph will be updated and the testing process will adapt.

### **2.3.1 OVERVIEW OF THE PROPOSED SYSTEM**

Correlix AI consists of interconnected modules that operate as a continuous penetration-testing workflow.

#### **1. Target and Scope Definition**

The system starts with a clear target, and a clear testing scope. If the assessment boundaries are configured, then testing activities will only occur within the configured boundaries.

#### **2. Reconnaissance and Observation Collection**

Authorized reconnaissance operations gather the knowledge of assets, services, configurations, vulnerabilities and other pertinent security observations.

The reconnaissance stage collects available information about the target, including:

- IP addresses and network ranges
- Hostnames and domains
- Open ports
- Running services
- Service versions
- Operating systems
- Network protocols
- Web applications and endpoints
- Technologies and software identified
- DNS and related domain information
- Discovered subdomains

- Network relationships

The collected information forms the initial security state of the target.

### **3. Vulnerability and Security Assessment**

Security assessment activities collect information such as:

- Identified vulnerabilities
- Vulnerability identifiers and severity
- Affected assets and services
- Misconfigurations
- Weak security settings
- Exposed services
- Authentication-related findings
- Web application security findings
- Configuration and version-related weaknesses
- Evidence associated with discovered findings

These observations are correlated with previously discovered assets and services.

### **4. Security Tool Orchestration**

A number of authorized security tools are integrated and managed through a shared workflow. The outputs of the tools are gathered and normalized to give consistent representation of observations across different tools.

The relevant information, like the source tool, the type of observation, the asset being observed, the time of observation, the severity of the observation, the evidence, and associated finding, is preserved for further processing.

## **5. Security State Management**

**The system maintains an evolving representation of the target using collected observations.**

**The security state may contain:**

- **Assets and hosts**
- **Services and ports**
- **Applications**
- **Technologies and versions**
- **Vulnerabilities**
- **Configurations**
- **Credentials or access conditions identified during authorized testing**
- **Relationships between discovered entities**
- **Testing observations**
- **Evidence**
- **Timestamps and observation history**

**New observations can add, modify, or update information in the current security state.**

## **6. Live Temporal Attack Graph**

A live temporal attack graph is used to represent the collected security information. Assets, services, vulnerabilities, conditions, and access states can be represented as graph entities, and relationships can specify a dependency or potential transition between the entities.

The graph also preserves temporal information so that if the security state changes during the assessment, it can be track.

## **7. Attack-Path Analysis**

The system examines the current graph for potential attack paths, by examining relationships within the graph. Path analysis is a method in which the current vulnerabilities, conditions, relationships, and evidence given are treated together and not individually.

Once the possible paths are identified, they can be prioritized for further security evaluation.

## **8. AI-Assisted Decision Engine**

The decision engine takes the current security state, observations available, attack-graph information and identified attack paths, to help select the next appropriate and authorized testing action.

The assessment scope and testing policies are limited, and thus the decision process is limited.

## **9. Controlled Action Execution**

Before execution, the selected action is checked against the authorized testing scope. Once approved, the relevant security tool or assessment procedure is executed and its output is collected as new evidence.

## **10. Feedback and Replanning**

Results from executed actions are returned to the system and used to update the security state and temporal attack graph.

The updated information is then used for further attack-path analysis and subsequent action selection.

The overall workflow is:

**Scope Definition → Reconnaissance → Information Collection → Vulnerability Assessment → Observation Normalization → Security State Update → Temporal Attack Graph Update → Attack-Path Analysis → AI-Assisted Decision → Controlled Action Execution → Feedback → Replanning**

This process continues until the defined assessment objectives or stopping conditions are reached.

### **2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM**

The proposed Correlix AI system has the following objectives:

- 1 To automate authorized reconnaissance and security assessment activities.
- 2 To collect information about assets, services, applications, vulnerabilities, configurations, and relationships.
- 3 To coordinate multiple security assessment tools through a unified workflow.
- 4 To normalize and correlate observations obtained from different tools.
- 5 To maintain an evolving representation of the target's security state.
- 6 To construct and continuously update a live temporal attack graph.

- 7 To identify and prioritize potential attack paths using available evidence.
- 8 To support AI-assisted selection of subsequent testing actions.
- 9 To use testing results as feedback for continuous replanning.
- 10 To maintain relevant evidence, observations, timestamps, and testing activity records.
- 11 To evaluate the proposed platform using defined test cases and suitable security assessment metrics.

### **2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM**

#### **1. Comprehensive Information Collection**

The platform combines reconnaissance, service, application, vulnerability, configuration, and relationship information into a unified security state.

#### **2. Automated Tool Coordination**

Multiple security assessment tools can be coordinated through a common workflow instead of being operated as completely isolated components.

#### **3. Improved Finding Correlation**

Observations from different assessment stages can be connected to the relevant assets, services, vulnerabilities, and relationships.

#### **4. Live Temporal Attack Graph**

The attack graph is continuously updated as new observations are obtained, providing a time-aware representation of the evolving security state.

## **5. Adaptive Testing**

Subsequent testing actions can be selected using newly discovered information rather than relying entirely on a fixed sequence.

## **6. Continuous Feedback**

Results from executed actions are incorporated into the security state and used for further analysis and replanning.

## **7. Attack-Path Visibility**

The system provides a structured representation of how individual findings may contribute to potential attack paths.

## **8. Traceable Assessment**

Observations, evidence, actions, graph updates, and decisions can be retained to support analysis and reporting.

## **9. Controlled Automation**

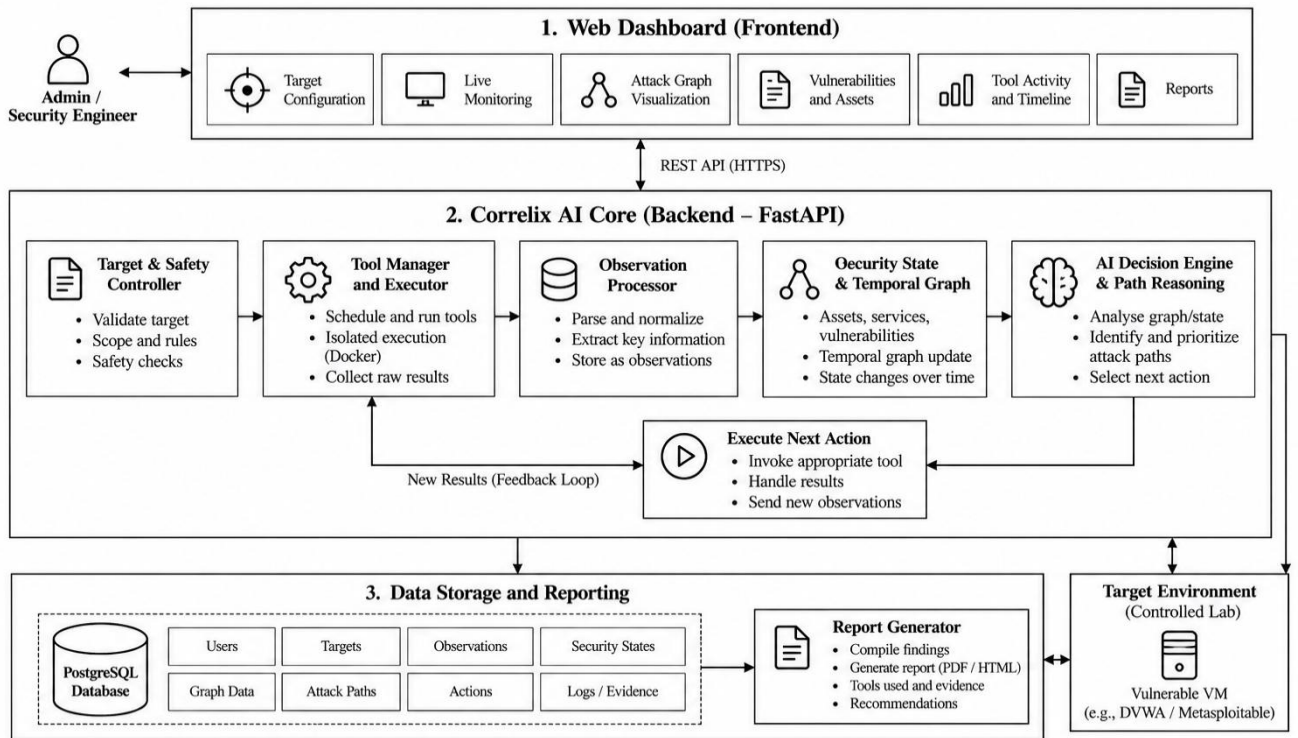
Automated testing remains restricted to explicitly authorized targets and defined assessment boundaries.

# CHAPTER 3

## SYSTEM DESIGN

### 3.1 SYSTEM ARCHITECTURE

Figure 3.1: System Architecture of Correlix AI



The Correlix AI system architecture integrates target and scope management, reconnaissance, security-tool orchestration, observation processing, security-state management, and a live temporal attack graph. Attack-path analysis and the AI decision engine determine suitable testing actions. The action executor performs authorized activities and feeds results back into the system, enabling continuous state updates, graph refinement, adaptive decision-making, and replanning throughout the penetration-testing process.

## **3.2 MODULE DESIGN**

### **3.2.1 Target & Scope Manager**

This module defines and validates the authorized testing scope. It manages target assets, permitted activities, boundaries, and testing constraints to ensure that all subsequent actions remain within the approved environment.

### **3.2.2 Reconnaissance & Information Collection**

This module collects information about the authorized target, including hosts, ports, services, applications, technologies, and network relationships. The collected information provides the initial data required for security assessment and state construction.

### **3.2.3 Security Tool Orchestrator**

This module coordinates multiple authorized security assessment tools according to the current testing requirements. It manages tool execution, collects their outputs, and forwards the results for further processing and correlation.

### **3.2.4 Observation Collector & Normalizer**

This module processes outputs received from security tools and other assessment activities. It extracts relevant observations, removes inconsistencies, and converts different outputs into a common format for further analysis.

### **3.2.5 Security State Manager**

This module maintains the current representation of the target's security state. It stores discovered assets, services, vulnerabilities, configurations, relationships, evidence, and timestamps, while updating the state as new observations are received.

### **3.2.6 Live Temporal Attack Graph**

This module represents the relationships between assets, services, vulnerabilities, conditions, and potential attack paths. Temporal information is maintained to reflect changes in the security state as the assessment progresses.

### **3.2.7 Attack-Path Analysis**

This module analyzes the live temporal attack graph to identify and prioritize potential attack paths. It considers discovered vulnerabilities, relationships, conditions, and available evidence to determine relevant paths for further assessment.

### **3.2.8 AI Decision Engine**

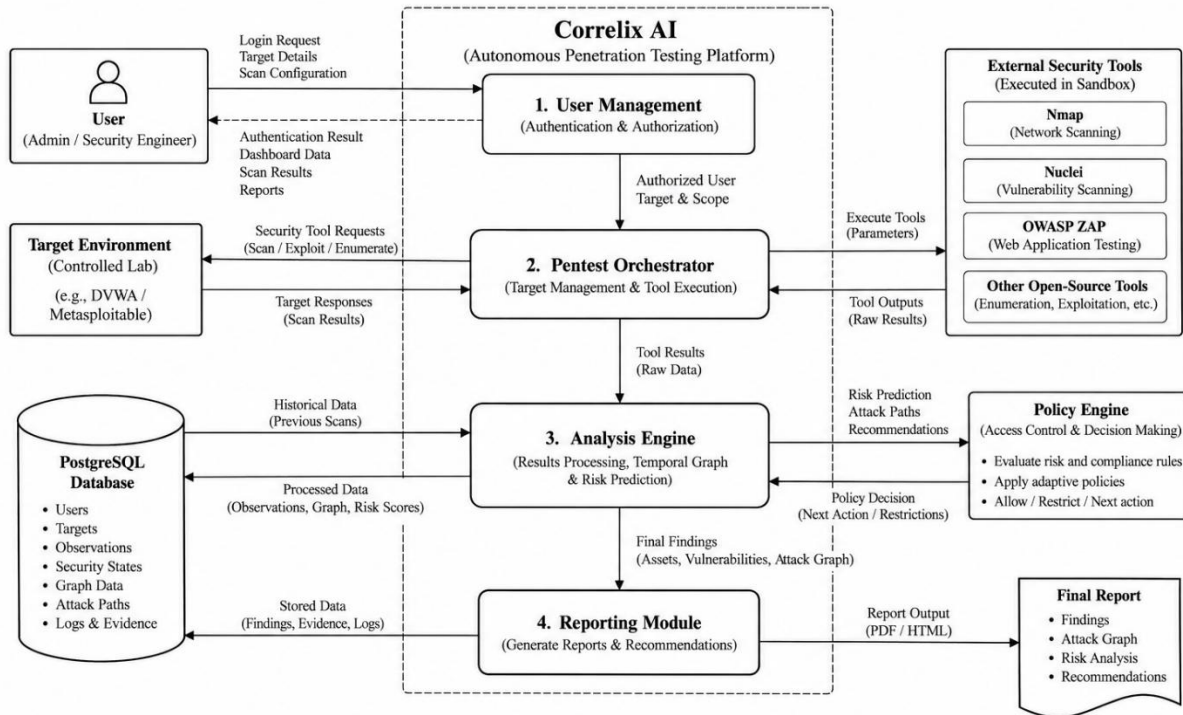
This module supports the selection of subsequent authorized testing actions. It considers the current security state, attack paths, observations, and testing constraints to determine an appropriate next action.

### **3.2.9 Action Executor**

This module executes the selected authorized testing action within the defined scope. The resulting output is collected as evidence and returned to the observation-processing stage, enabling security-state updates and continuous replanning.

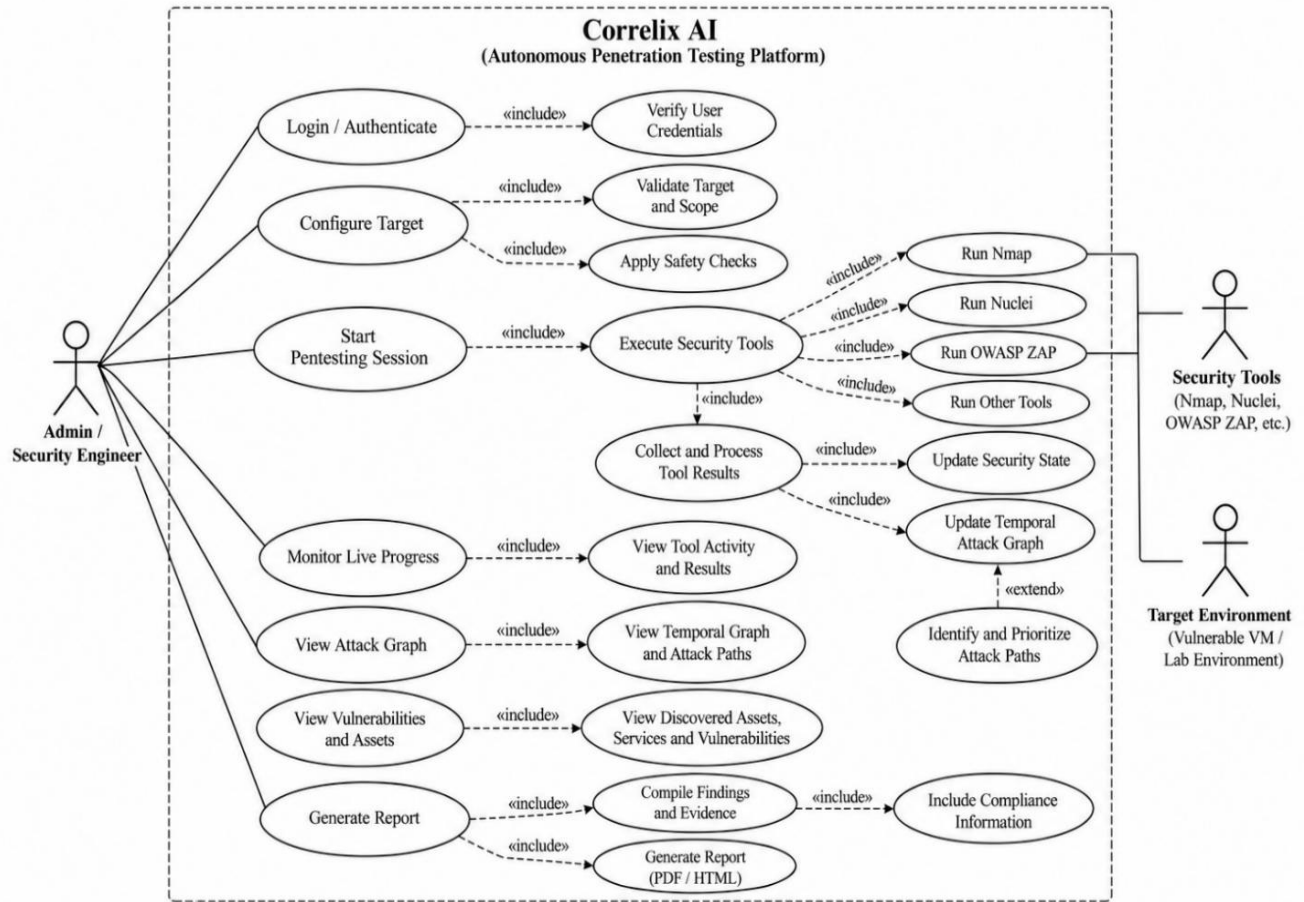
### 3.3 DATA FLOW DIAGRAM

Figure 3.2: Data Flow Diagram of Correlix AI



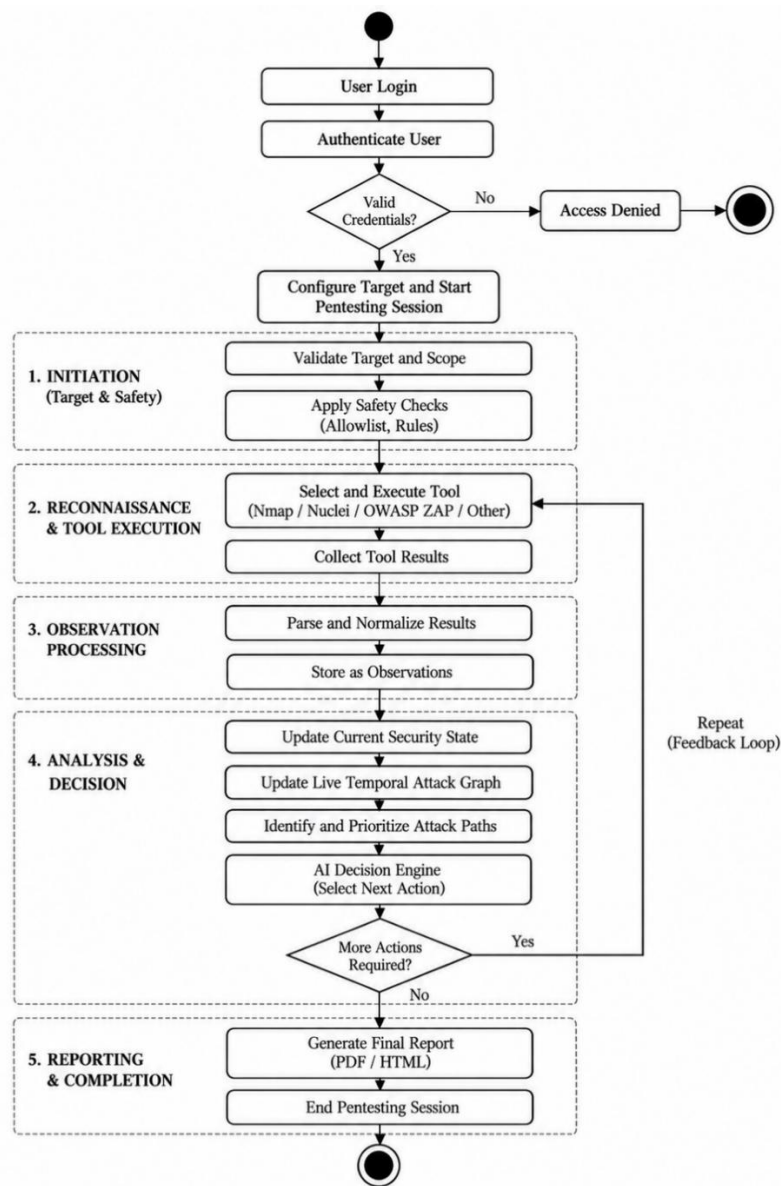
The Data Flow Diagram illustrates how information moves through Correlix AI. User requests and target details are processed by the platform, which coordinates security tools and collects their results. The Analysis Engine processes observations, updates the temporal attack graph, and evaluates risks. The processed findings are stored in the database and used to generate the final security report.

**Figure 3.3: Use Case Diagram of Correlix AI**

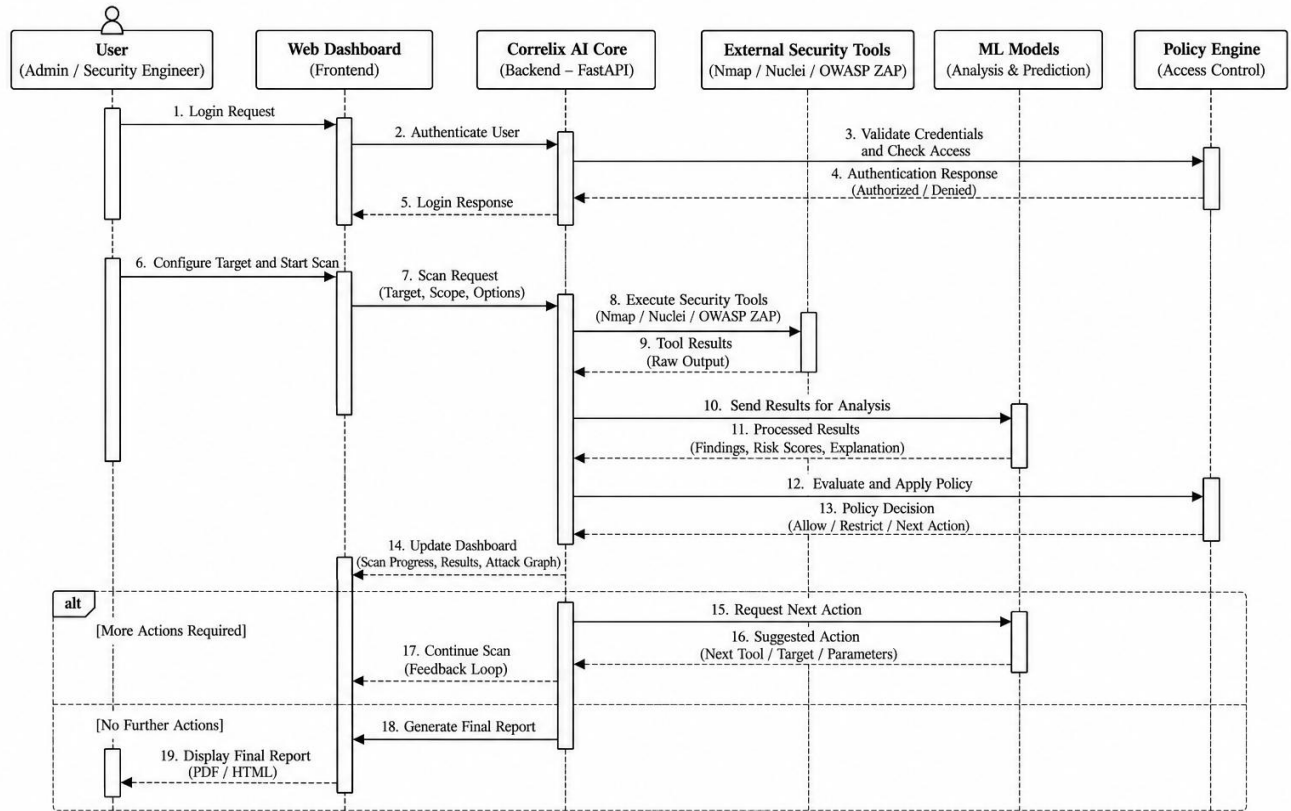


The Use Case Diagram represents the interactions between the Admin/Security Engineer, security tools, and target environment. The user can authenticate, configure targets, start testing sessions, monitor progress, view vulnerabilities and attack graphs, and generate reports. The system executes security tools, processes results, updates the security state and temporal graph, and identifies potential attack paths.

**Figure 3.4: Activity Diagram of Correlix AI**



The Activity Diagram shows the complete penetration-testing workflow of Correlix AI. The process begins with user authentication and target configuration, followed by scope validation and safety checks. Security tools are executed and their results are processed. The security state and temporal attack graph are updated, attack paths are analyzed, and the AI selects further actions through a feedback loop before generating the final report.



**Figure 3.5: Sequence Diagram of Correlix AI**

The Sequence Diagram describes the chronological interaction between the user, web dashboard, Correlix AI core, security tools, ML models, and policy engine. It shows authentication, target configuration, tool execution, result processing, policy evaluation, and dashboard updates. When further actions are required, the system selects the next action and continues the feedback loop until the final report is generated.

## 3.4 SECURITY CONTROLS

The security controls in Correlix AI are designed to ensure that automated penetration-testing activities remain **authorized, controlled, traceable, and within the defined testing scope**. The controls are applied throughout authentication, target configuration, tool execution, decision-making, and reporting.

### 3.4.1 User Authentication and Authorization

This control verifies the identity of the user before providing access to Correlix AI. Only authorized users can configure targets, initiate penetration-testing sessions, monitor activities, and

access assessment results. Authorization also ensures that available actions are restricted according to the user's permitted access level.

### **3.4.2 Target and Scope Validation**

This control verifies the target environment and defined testing scope before any security assessment activity begins. It ensures that testing is performed only against approved assets and prevents actions from being directed outside the authorized environment.

### **3.4.3 Safety and Execution Controls**

Safety controls are applied before executing security tools or testing actions. The system checks predefined rules, allowed targets, and testing restrictions before execution. Actions that do not satisfy the defined conditions can be restricted or stopped.

### **3.4.4 Policy-Based Action Control**

The Policy Engine evaluates proposed actions based on defined security and access policies. It can allow, restrict, or reject a requested action. This provides an additional control layer before subsequent automated testing activities are performed.

### **3.4.5 Tool Execution Isolation**

External security tools are executed within a controlled testing environment. This limits the impact of automated tool execution and separates security assessment activities from unintended environments or resources.

### **3.4.6 Audit Logging and Evidence Protection**

Correlix AI maintains records of testing activities, observations, tool outputs, findings, and supporting evidence. These records provide traceability of assessment activities and support analysis, verification, and final report generation.

### **3.4.7 Feedback and Action Verification**

Results from each executed action are processed before further actions are selected. The updated security state, attack graph, and applicable policies are considered when determining the next action. This prevents the system from continuing through a completely uncontrolled sequence of automated activities.

# CHAPTER 4

## SYSTEM DESCRIPTION

### 4.1 DATA COLLECTION

Correlix AI collects security information from the authorized target environment through reconnaissance activities and integrated security assessment tools. The collected information is used to build the current security state, identify relationships, construct the live temporal attack graph, and support attack-path analysis and AI-assisted decision-making.

#### Data Categories

**Table 4.1 Data Categories**

| Category                  | Examples of Data                                                   |
|---------------------------|--------------------------------------------------------------------|
| Target Information        | IP addresses, hostnames, domains, subdomains                       |
| Network Information       | Network ranges, protocols, network relationships                   |
| Service Information       | Open ports, services, service versions                             |
| System Information        | Operating systems, software, technologies                          |
| Application Information   | Web applications, endpoints, application technologies              |
| Vulnerability Information | Vulnerabilities, severity, affected assets                         |
| Configuration Information | Misconfigurations, exposed settings, security conditions           |
| Access Information        | Discovered access conditions and privileges                        |
| Relationship Information  | Asset-service, service-vulnerability, and dependency relationships |
| Attack-Path Information   | Potential paths, path conditions, affected assets                  |
| Tool Information          | Tool output, scan results, source tool                             |
| Evidence                  | Findings, observations, supporting assessment data                 |
| Temporal Information      | Observation timestamps, discovery history, state changes           |

The collected information is normalized and associated with the relevant target entities. New observations obtained during testing can update the current security state and temporal attack graph.

## **4.2 DATA PREPROCESSING**

The proposed Correlix AI methodology combines **security-tool orchestration, observation correlation, security-state management, temporal attack-graph construction, attack-path analysis, and AI-assisted action selection.**

### **4.2.1 Overall Methodology**

The methodology follows:

**Data Collection → Preprocessing → Security State Update → Temporal Attack Graph → Attack-Path Analysis → AI Decision → Action Execution → New Observation → Replanning**

#### **A. Observation Collection and Normalization**

Security tools provide information about assets, services, applications, vulnerabilities, configurations, and other security observations.

The collected outputs are parsed and normalized into a common representation so that findings from different tools can be correlated.

#### **B. Security State Management**

The normalized observations are used to maintain the current security state of the target.

The security state contains relevant information such as:

- Assets and hosts

- Services and ports
- Applications
- Vulnerabilities
- Configurations
- Relationships
- Evidence
- Observation timestamps

New observations can add or modify information in the current state.

### **C. Live Temporal Attack Graph**

The security state is represented using a **live temporal attack graph**.

The graph contains nodes representing entities such as assets, services, vulnerabilities, and conditions. Edges represent relevant relationships and potential transitions between entities.

Temporal information is maintained so that changes in the security state and newly discovered relationships can be reflected in the graph.

### **D. Attack-Path Analysis**

The current attack graph is analyzed to identify possible attack paths.

The analysis considers available vulnerabilities, relationships, conditions, and supporting evidence. Identified paths can be prioritized based on their relevance to the current security state.

## **E. AI-Assisted Decision-Making**

The AI decision engine uses the current security state and attack-path information to support selection of the next authorized testing action.

The decision mechanism may consider:

- Current discovered information
- Relevant vulnerabilities
- Available attack paths
- Previous testing results
- Potential value of the next action
- Defined testing constraints

The final decision mechanism and prioritization criteria will be validated through experimentation.

## **F. Action Execution**

The selected testing action is passed to the appropriate security tool or execution component.

The action is performed only within the defined authorization scope, and its output is collected as new evidence.

## **G. Continuous Feedback and Replanning**

The results of each executed action are returned to the observation-processing stage.

The system then:

1. Collects the new observation.
2. Updates the security state.
3. Updates the temporal attack graph.
4. Re-evaluates potential attack paths.
5. Selects the next relevant testing action.
6. Repeats the process until the assessment objectives or stopping conditions are reached.

This creates a continuous adaptive loop:

**Observe → Update State → Update Graph → Analyze Paths → Select Action → Execute → Observe Again**

#### 4.2.2 RESULTS AND DISCUSSION

The results section will present the **actual experimental outcomes obtained from the implemented Correlix AI platform**. Since the final implementation and experiments have not yet been completed, numerical results should not be inserted at this stage.

The evaluation will focus on the ability of the proposed system to collect, correlate, represent, and use security information during an authorized penetration test.

**Table 4.2: Evaluation category and metrics**

| <b>Evaluation Category</b> | <b>Metrics</b>                                     |
|----------------------------|----------------------------------------------------|
| Reconnaissance             | Asset and service discovery accuracy               |
| Vulnerability Assessment   | Finding accuracy and detection rate                |
| Observation Processing     | Parsing and normalization accuracy                 |
| Attack Graph               | Graph update correctness and relationship accuracy |
| Attack-Path Analysis       | Path identification and prioritization             |

|                     |                                            |
|---------------------|--------------------------------------------|
| AI Decision-Making  | Action selection effectiveness             |
| Adaptability        | Response to newly discovered information   |
| Execution           | Action success and execution time          |
| System Performance  | Processing time and computational overhead |
| Evidence Management | Completeness and traceability of records   |

### Expected Analysis

The experimental analysis should investigate:

- Whether the platform correctly collects and correlates security observations.
- Whether the temporal attack graph accurately reflects changes in the target security state.
- Whether relevant attack paths can be identified and prioritized.
- Whether AI-assisted decisions adapt to newly discovered information.
- Whether feedback-based replanning improves the testing workflow.
- What computational overhead is introduced by continuous graph updates and decision-making.

### Results Table

The dashboard after processing will include these:

**Table 4.3 Result Table**

| Result Area       | Evidence to Present                       |
|-------------------|-------------------------------------------|
| Target Validation | Screenshot of configured target and scope |
| Tool Execution    | Tool execution/activity output            |
| Asset Discovery   | Discovered hosts, ports and services      |

|                          |                                           |
|--------------------------|-------------------------------------------|
| Vulnerability Assessment | Identified vulnerabilities and severity   |
| Observation Processing   | Normalized observation records            |
| Security State           | Current asset/service/vulnerability state |
| Temporal Attack Graph    | Live attack graph screenshot              |
| Attack-Path Analysis     | Identified/prioritized attack paths       |
| AI Decision              | Selected next testing action              |
| Feedback Loop            | Before/after graph or state update        |
| Evidence Storage         | Stored logs and assessment evidence       |
| Reporting                | Generated penetration-testing report      |

## Discussion

The proposed architecture integrates reconnaissance, observation processing, security-state management, temporal attack-graph construction, attack-path analysis, and AI-assisted action selection. The feedback mechanism enables newly obtained results to update the security state and guide subsequent testing. This provides an adaptive and traceable approach to authorized penetration testing.

## CHAPTER 5

# CONCLUSION

**Correlix AI** proposes an autonomous penetration-testing platform that integrates reconnaissance, security-tool orchestration, observation processing, security-state management, live temporal attack graphs, attack-path analysis, and AI-assisted decision-making into a unified framework.

The platform continuously collects information about assets, services, applications, vulnerabilities, configurations, relationships, and testing evidence. These observations are processed and used to maintain an evolving security state. The live temporal attack graph represents the relationships between discovered security elements and helps identify potential attack paths as new information becomes available.

The AI-assisted decision mechanism uses the current security state, observations, and identified attack paths to support the selection of suitable subsequent testing actions. Results from executed actions are fed back into the system, allowing the security state and attack graph to be updated and enabling the testing process to adapt to newly discovered information.

Overall, Correlix AI aims to provide an **adaptive, context-aware, and traceable approach to authorized penetration testing**. The proposed system will be evaluated in a controlled environment using suitable test cases and metrics to assess its effectiveness in security assessment, attack-path analysis, adaptive decision-making, and continuous replanning.

## REFERENCES

1. C. Skandylas and M. Asplund, “Automated Penetration Testing: Formalization and Realization,” *Computers & Security*, vol. 155, Art. no. 104454, 2025, doi: 10.1016/j.cose.2025.104454.
2. J. L. Obes, C. Sarraute, and G. Richarte, “Attack Planning in the Real World,” 2013.
3. X. Shen, L. Wang, Z. Li, Y. Chen, W. Zhao, D. Sun, J. Wang, and W. Ruan, “PentestAgent: Incorporating LLM Agents to Automated Penetration Testing,” in *Proc. 20th ACM Asia Conf. Comput. Commun. Security (ASIA CCS)*, 2025, pp. 375–391, doi: 10.1145/3708821.3733882.
4. H. Kong, D. Hu, J. Ge, L. Li, T. Li, and B. Wu, “VulnBot: Autonomous Penetration Testing for A Multi-Agent Collaborative Framework,” 2025.
5. Q. Li, R. Wang, D. Li, F. Shi, M. Zhang, and A. Chattopadhyay, “DynPen: Automated Penetration Testing in Dynamic Network Scenarios Using Deep Reinforcement Learning,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 8966–8981, 2024, doi: 10.1109/TIFS.2024.3461950.
6. J. Gu, L. Zhang, and Z. Li, “MAPT: Memory-Augmented Multi-Agent Framework for Automated Penetration Testing,” in *Proc. 2026 6th Int. Symp. Comput. Technol. Inf. Sci. (ISCTIS)*, 2026, doi: 10.1109/ISCTIS70043.2026.11572126.
7. “AutoPentestAL: An Automated Penetration Testing Framework with Active Learning Capabilities,” in *Proc. IEEE TrustCom*, 2025, doi: 10.1109/Trustcom66490.2025.00041.
8. “SHARKAPT: An Autonomous LLM-Orchestrated Penetration Testing Framework with MCP-Based Tool Integration,” in *Proc. IEEE JCSSE*, 2026, doi: 10.1109/JCSSE68839.2026.11596758.
9. Z. Chen, C. Yi, and P. Jia, “Agentic RAG for Command Generation in Automated Penetration Testing,” in *Proc. 2025 Int. Conf. Signal Process., Comput. Netw. Commun. (SPCNC)*, 2025, pp. 579–583, doi: 10.1109/SPCNC68200.2025.11406223.
10. M. Somma, M. Großpointner, P. Zabalegui, E. Heilimo, and B. Stojanović, “Environment-Grounded Multi-Agent Workflow for Autonomous Penetration Testing,” in *Proc. 2026*

- IEEE 12th Int. Conf. Network Softwarization (NetSoft), 2026, pp. 570–576, doi: 10.1109/NetSoft70012.2026.11603445.
11. S. C. Dinesh, D. Selvam, V. S. Yallal, and S. V. S. Suvedya, “RedOps AI: Multi-Agent Orchestration for Autonomous Penetration Testing,” in Proc. 4th IEEE Int. Conf. Knowledge Engineering and Communication Systems (ICKECS), 2026, doi: 10.1109/ICKECS70176.2026.11527848.
  12. F. S. A. Abusalama, M. Y. Alsherbiny, L. A. K. Alshalabi, M. R. M. R. Akchar, and M. Ilyas, “CyberPentest: An Automated Penetration Testing Framework Powered by LLMs,” in Proc. 2025 26th Int. Arab Conf. Inf. Technol. (ACIT), 2025, pp. 498–501, doi: 10.1109/ACIT68900.2025.11510561.
  13. S. Siddiq, R. B. Hadiprakoso, P. A. W. Putro, G. Girinoto, and D. F. Saputra, “Pungoe Pentest: Orchestration of Web Penetration Testing Tools with Assistance Based on Large Language Model (LLM) Pentest-AI and Retrieval-Augmented Generation (RAG),” in Proc. 2025 IEEE 2nd Int. Conf. Cryptography, Informatics and Cybersecurity (ICoCICs), 2025, doi: 10.1109/ICoCICs68032.2025.11384036.
  14. L. A. K. Alshalabi, M. Y. Alsherbiny, F. S. A. Abusalama, A. M. J. Charedeh, and M. Ilyas, “AI-Assisted Automation of Penetration Testing Using Local LLMs and RAG,” in Proc. 2025 26th Int. Arab Conf. Inf. Technol. (ACIT), 2025, doi: 10.1109/ACIT68900.2025.11510657.
  15. G. Sánchez, O. Olayinka, and A. Pasikhani, “Web Application Penetration Testing with Artificial Intelligence: A Systematic Review,” in Proc. IEEE 23rd Int. Conf. Cyber Security and Resilience, 2024, pp. 236–245, doi: 10.1109/NCA61908.2024.00043.
  16. “Web Penetration Testing Using Collaborative Multi-Agents and Integrated Reporting,” in Proc. IEEE ICADEIS, 2025, doi: 10.1109/ICADEIS65852.2025.10933244.
  17. N. P. Kumar, V. K. G. K. Kumar, and D. M. S. Dhanushree, “AIPenTool: A Unified Approach to Automated Penetration Testing for Enhanced Network and Web Application Security,” in Proc. 2025 Int. Conf. Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE), 2025, doi: 10.1109/IITCEE64140.2025.10915305.
  18. S. G. Bianou and R. G. Batogna, “PENTEST-AI, an LLM-Powered Multi-Agents Framework for Penetration Testing Automation Leveraging MITRE ATT&CK,” in Proc.

IEEE Int. Conf. Cyber Security and Resilience (CSR), 2024, pp. 763–770, doi: 10.1109/CSR61664.2024.10679480.

19. Y. Wang, C. Huo, S. Wang, M. He, and Y. Liu, “Design Methodology for an Automated Penetration Testing System Based on Improved Monte Carlo Tree Search,” in Proc. Int. Conf. Information Processing and Network Provisioning (ICIPNP), 2023, doi: 10.1109/ICIPNP62754.2023.00100.
20. A. Confido, E. V. Ntagiou, and M. Wallum, “Reinforcing Penetration Testing Using AI,” in Proc. IEEE Aerospace Conf. (AERO), 2022, doi: 10.1109/AERO53065.2022.9843459.
21. “AutoPT: How Far Are We From the Fully Automated Web Penetration Testing?” IEEE Transactions on Information Forensics and Security, vol. 20, pp. 9657–9672, 2025, doi: 10.1109/TIFS.2025.3601343.